

Giới thiệu Pandas

DataFrame, Series, làm sạch dữ liệu, gom nhóm và phân tích dữ liệu kinh doanh

Duc-Minh Vu

SLSCM Lab - FDA - NEU

August 17, 2026

Mục tiêu học tập

Sau khi hoàn thành bài học, người học có thể:

- ▶ Giải thích vai trò của Pandas trong khoa học dữ liệu và phân tích kinh doanh.
- ▶ Phân biệt Series và DataFrame.
- ▶ Tạo, kiểm tra, chọn, lọc và sắp xếp dữ liệu dạng bảng.
- ▶ Đọc và ghi dữ liệu CSV, Excel, JSON và tệp văn bản.
- ▶ Phát hiện và xử lý giá trị thiếu, dòng trùng lặp và kiểu dữ liệu không nhất quán.
- ▶ Biến đổi cột và áp dụng các hàm đơn giản.
- ▶ Sử dụng `groupby()`, `agg()`, `merge()`, `concat()` và pivot table.
- ▶ Tính thống kê mô tả, tương quan và một số tổng hợp chuỗi thời gian đơn giản.

Lộ trình bài học

- 1 Vì sao cần học Pandas?
- 2 Pandas là gì?
- 3 Pandas trong quy trình Data Science
- 4 Thiết lập môi trường
- 5 DataFrame đầu tiên
- 6 Series và DataFrame
- 7 Đọc lệnh Pandas
- 8 Kiểm tra, chọn, lọc và sắp xếp dữ liệu
- 9 Đọc và ghi tệp dữ liệu
- 10 Làm sạch dữ liệu
- 11 Tạo cột, gom nhóm và tổng hợp
- 12 Merge, concat và pivot table
- 13 Tương quan, trực quan hóa, chuỗi thời gian
- 14 Bài tập thực hành

Vì sao cần học Pandas?

Pandas là cầu nối thực hành giữa dữ liệu thô và các mô hình khoa học dữ liệu.

- ▶ Dữ liệu thực tế thường lộn xộn, thiếu giá trị và nằm ở nhiều định dạng khác nhau.
- ▶ Pandas giúp tổ chức bản ghi thành dòng và thuộc tính thành cột.
- ▶ Pandas hỗ trợ kiểm tra, làm sạch, biến đổi, tổng hợp và báo cáo dữ liệu.
- ▶ Pandas làm việc tốt với NumPy, Matplotlib, Scikit-learn và Statsmodels.

Quy trình phân tích kinh doanh

Dữ liệu thô → bảng dữ liệu sạch → insight tổng hợp → dữ liệu sẵn sàng cho mô hình.

Pandas là gì?

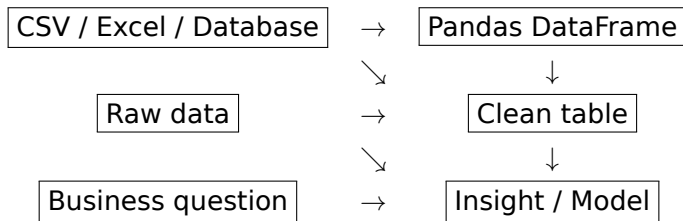
Pandas là một thư viện Python mã nguồn mở dùng cho **xử lý và phân tích dữ liệu**. Thư viện này được xây dựng trên NumPy và cung cấp các công cụ cấp cao để làm việc với dữ liệu có cấu trúc và dữ liệu dạng bảng.

Pandas đặc biệt hữu ích cho:

- ▶ đọc dữ liệu từ CSV, Excel, JSON và tệp văn bản;
- ▶ làm sạch, biến đổi và chuẩn bị dữ liệu;
- ▶ chọn, lọc, gom nhóm và tổng hợp các quan sát;
- ▶ kết hợp nhiều tập dữ liệu;
- ▶ tính thống kê mô tả và trực quan hóa nhanh.

Pandas trong quy trình Khoa học dữ liệu

Pandas thường nằm ở bước chuyển dữ liệu thô thành dữ liệu sạch và có thể phân tích được.



- ▶ NumPy hỗ trợ tính toán số học hiệu quả.
- ▶ Pandas tổ chức dữ liệu theo bảng: dòng, cột và chỉ số.
- ▶ Matplotlib/Seaborn trực quan hóa kết quả từ DataFrame.
- ▶ Scikit-learn thường nhận dữ liệu đã được xử lý từ Pandas.

Yêu cầu nền tảng và thiết lập môi trường

Kiến thức nên có

- ▶ Python cơ bản: biến, list, dictionary, vòng lặp và hàm.
- ▶ Kiến thức NumPy cơ bản là một lợi thế.
- ▶ Môi trường làm việc: Jupyter Notebook, JupyterLab, Google Colab, VS Code hoặc công cụ tương tự.

Cài đặt và import

```
pip install pandas
```

```
import pandas as pd  
print(pd.__version__)
```

Tạo một DataFrame

```
import pandas as pd

data = {
    "Name": ["An", "Binh", "Chi"],
    "Age": [20, 21, 19],
    "GPA": [3.2, 3.6, 3.4]
}

df = pd.DataFrame(data)
print(df)
```

Diễn giải

Mỗi khóa trong dictionary trở thành một cột. Mỗi list cung cấp các giá trị cho cột đó.

Kết quả chạy code: DataFrame đầu tiên

Khi in df, Pandas hiển thị bảng gồm 3 dòng và 3 cột.

	Name	Age	GPA
0	An	20	3.2
1	Binh	21	3.6
2	Chi	19	3.4

Điểm cần chú ý

Cột bên trái 0, 1, 2 là index mặc định do Pandas tự tạo.

Cấu trúc của DataFrame

Thành phần	Ý nghĩa
Dòng	Bản ghi, quan sát, sinh viên, khách hàng, sản phẩm, đơn hàng.
Cột	Biến, thuộc tính, trường dữ liệu, đặc trưng.
Index	Nhãn dòng dùng để định danh và chọn dữ liệu.
Nhãn cột	Tên dùng để truy cập các biến.
Giá trị	Dữ liệu thực tế nằm bên trong bảng.

Ví dụ

Một bảng sinh viên có thể gồm các cột StudentID, Name, Age, Major và GPA.

Series và DataFrame từ ví dụ đầu tiên

Một cột thường là Series

```
print(sales["Price"])
```

```
0    10
1    20
2     8
Name: Price, dtype: int64
```

Nhiều cột là DataFrame

```
print(sales[["Product", "Price"]])
```

	Product	Price
0	A	10
1	B	20
2	C	8

Ghi nhớ

Một cột có nhãn chỉ số là Series; một bảng hai chiều là DataFrame.

Cách đọc lệnh Pandas

Phần lớn lệnh Pandas có một trong hai dạng sau:

`pd.function(input, option=value)` hoặc `df.method(option=value)`

- ▶ `pd`: bí danh chuẩn của thư viện Pandas.
- ▶ `df`: biến thường dùng để lưu một DataFrame.
- ▶ `function`: hàm cấp thư viện, ví dụ `pd.read_csv()`.
- ▶ `method`: thao tác gắn với một DataFrame hoặc Series.
- ▶ `option=value`: tham số đặt tên dùng để điều khiển hành vi của lệnh.

Ví dụ

`pd.read_csv("sales.csv")` đọc tệp CSV và đưa dữ liệu vào một DataFrame.

Mẫu lệnh: đối tượng, thuộc tính, phương thức, hàm

Dạng lệnh	Ý nghĩa
<code>pd.DataFrame(data)</code>	Gọi hàm Pandas để tạo một bảng dữ liệu.
<code>df.shape</code>	Đọc thuộc tính mô tả số dòng và số cột.
<code>df.head()</code>	Gọi phương thức gắn với DataFrame hiện có.
<code>pd.merge(left, right)</code>	Gọi hàm Pandas để kết hợp hai bảng dữ liệu.
<code>df["Sales"].mean()</code>	Chọn một cột rồi gọi phương thức của Series.

Ghi chú giảng dạy

Với mỗi lệnh, sinh viên nên xác định ba thành phần: đầu vào, thao tác và đầu ra.

Ví dụ nhỏ: giải thích từng dòng

```
import pandas as pd

sales = pd.DataFrame({
    "Product": ["A", "B", "C"],
    "Quantity": [2, 3, 5],
    "Price": [10, 20, 8]
})

sales["Revenue"] = sales["Quantity"] * sales["Price"]
print(sales)
```

- ▶ `pd.DataFrame(...)` tạo một tập dữ liệu dạng bảng.
- ▶ Mỗi khóa trong dictionary trở thành tên cột.
- ▶ `sales["Revenue"] = ...` tạo một cột tính toán mới.
- ▶ Phép nhân được thực hiện theo từng dòng trên hai cột đã chọn.

Kết quả chạy code: ví dụ doanh thu

Sau khi chạy ví dụ ở slide trước, cột Revenue được tạo bằng $\text{Quantity} * \text{Price}$.

	Product	Quantity	Price	Revenue
0	A	2	10	20
1	B	3	20	60
2	C	5	8	40

Cách đọc kết quả

Mỗi dòng là một sản phẩm. Ví dụ, sản phẩm B có $\text{Quantity} = 3$, $\text{Price} = 20$, nên $\text{Revenue} = 60$.

Quick Check: Pandas là gì?

Câu 1. Bí danh chuẩn thường dùng cho Pandas là gì?

- ▶ A. pn
- ▶ B. pd
- ▶ C. ps
- ▶ D. pa

Câu 2. Cấu trúc dữ liệu nào của Pandas là hai chiều?

- ▶ A. Series
- ▶ B. tuple
- ▶ C. DataFrame
- ▶ D. ndarray

Kiểm tra nhanh DataFrame

```
print(df.head())  
print(df.tail())  
print(df.shape)  
print(df.columns)  
print(df.index)  
print(df.dtypes)  
df.info()  
print(df.describe())
```

Mục đích

Bước kiểm tra giúp trả lời các câu hỏi: có bao nhiêu dòng? có những cột nào? kiểu dữ liệu là gì? có giá trị thiếu không?

Ví dụ kết quả: kiểm tra nhanh DataFrame

Với DataFrame sinh viên ở trên, một số lệnh kiểm tra cho kết quả như sau:

```
print(df.shape)
# (3, 3)

print(df.columns)
# Index(['Name', 'Age', 'GPA'], dtype='object')

print(df.dtypes)
# Name      object
# Age       int64
# GPA       float64
# dtype: object
```

Cách đọc

shape = (3, 3) nghĩa là bảng có 3 dòng và 3 cột. Kiểu object thường dùng cho dữ liệu chuỗi.

Các lệnh chính: kiểm tra DataFrame

Lệnh	Ý nghĩa
<code>df.head()</code>	Hiển thị các dòng đầu tiên.
<code>df.tail()</code>	Hiển thị các dòng cuối cùng.
<code>df.shape</code>	Trả về số dòng và số cột.
<code>df.columns</code>	Trả về nhãn cột.
<code>df.index</code>	Trả về nhãn dòng.
<code>df.dtypes</code>	Trả về kiểu dữ liệu của từng cột.
<code>df.info()</code>	Tóm tắt cấu trúc DataFrame.
<code>df.describe()</code>	Thống kê mô tả cho các cột số.

Mini Exercise: tạo và kiểm tra dữ liệu

Tạo một DataFrame tên là products với các cột Product, Price và Stock.

```
products = pd.DataFrame({  
    "Product": ["Laptop", "Mouse", "Keyboard"],  
    "Price": [1200, 25, 45],  
    "Stock": [5, 50, 30]  
})  
  
print(products.head())  
print(products.shape)  
print(products.dtypes)
```

Kiểm tra

Tất cả các cột phải có cùng số lượng giá trị.

Kết quả chạy code: DataFrame sản phẩm

Khi chạy phần mini exercise, sinh viên có thể kỳ vọng kết quả như sau:

	Product	Price	Stock
0	Laptop	1200	5
1	Mouse	25	50
2	Keyboard	45	30

```
print(products.shape)  
# (3, 3)
```

```
print(products.dtypes)  
# Product      object  
# Price         int64  
# Stock         int64  
# dtype: object
```

Quick Check: cơ bản về DataFrame

Câu 1. Lệnh nào tạo một DataFrame?

- ▶ A. `pd.DataFrame()`
- ▶ B. `pd.SeriesFrame()`
- ▶ C. `np.DataFrame()`
- ▶ D. `df.create()`

Câu 2. Lệnh nào trả về số dòng và số cột?

- ▶ A. `df.shape`
- ▶ B. `df.mean()`
- ▶ C. `df.merge()`
- ▶ D. `df.plot()`

Tạo Series

Series là một mảng một chiều có nhãn.

```
s = pd.Series([10, 20, 30], index=["A", "B", "C"])  
print(s)  
print(s.index)  
print(s.values)  
print(s.dtype)
```

Diễn giải

Series giống như một cột có nhãn. Khi tính toán, Pandas căn chỉnh các Series theo nhãn index.

Kết quả chạy code: Series

```
A    10  
B    20  
C    30  
dtype: int64  
  
Index(['A', 'B', 'C'], dtype='object')  
[10 20 30]  
int64
```

Cách đọc

Cột bên trái là nhãn index; cột bên phải là giá trị của Series.

Căn chỉnh Series theo nhãn

```
x = pd.Series([10, 20, 30], index=["A", "B", "C"])
y = pd.Series([1, 2, 3], index=["B", "C", "D"])

result = x + y
print(result)
```

Ý tưởng dự kiến

Chỉ các nhãn khớp nhau mới được cộng trực tiếp. Các nhãn không khớp sẽ tạo ra giá trị thiếu.

Kết quả chạy code: căn chỉnh Series

```
A      NaN  
B      21.0  
C      32.0  
D      NaN  
dtype: float64
```

Vì sao có NaN?

Nhãn B và C xuất hiện trong cả hai Series nên được cộng. Nhãn A chỉ có trong x, còn D chỉ có trong y, nên kết quả là NaN.

Dữ liệu ví dụ cho lựa chọn, lọc và sắp xếp

Để các ví dụ loc, iloc, lọc và sắp xếp có kết quả rõ ràng, ta dùng bảng sinh viên sau.

```
df = pd.DataFrame({  
    "StudentID": ["S01", "S02", "S03", "S04"],  
    "Name": ["An", "Binh", "Chi", "Dung"],  
    "Age": [20, 21, 19, 22],  
    "GPA": [3.2, 3.6, 3.4, 3.8]  
})  
  
print(df)
```

	StudentID	Name	Age	GPA
0	S01	An	20	3.2
1	S02	Binh	21	3.6
2	S03	Chi	19	3.4
3	S04	Dung	22	3.8

Chọn cột

Một cột trả về Series

```
name = df["Name"]  
print(name)
```

Nhiều cột trả về DataFrame

```
subset = df[["Name", "GPA"]]  
print(subset)
```

Ghi chú cú pháp

Dùng một cặp ngoặc vuông để chọn một cột; dùng một list tên cột để chọn nhiều cột.

Kết quả chạy code: chọn cột

Một cột: Series

```
0      An
1     Binh
2     Chi
3     Dung
Name: Name, dtype: object
```

Nhiều cột: DataFrame

```
      Name  GPA
0      An  3.2
1     Binh  3.6
2     Chi  3.4
3     Dung  3.8
```

Chọn dòng bằng loc và iloc

Chọn theo nhãn

```
df = df.set_index("StudentID")  
print(df.loc["S01"])
```

Chọn theo vị trí

```
print(df.iloc[0])  
print(df.iloc[0:2])
```

Điểm khác biệt cốt lõi

loc dùng nhãn; iloc dùng vị trí số nguyên.

Kết quả chạy code: loc và iloc

Sau khi đặt StudentID làm index, loc có thể chọn theo mã sinh viên.

```
print(df.loc["S01"])  
# Name      An  
# Age       20  
# GPA       3.2  
# Name: S01, dtype: object  
  
print(df.iloc[0:2])  
#           Name  Age  GPA  
# StudentID  
# S01         An   20   3.2  
# S02        Binh   21   3.6
```

Chọn đồng thời dòng và cột

```
# ọchn dòng và ọct theo nhãn  
selected = df.loc[["S01", "S02"], ["Name", "GPA"]]  
print(selected)  
  
# ọchn dòng và ọct theo iv trí  
selected = df.iloc[0:2, [0, 2]]  
print(selected)
```

Cách đọc cú pháp

Phần trước dấu phẩy chọn dòng; phần sau dấu phẩy chọn cột.

Các lệnh chính: Series và lựa chọn dữ liệu

Lệnh	Ý nghĩa
<code>pd.Series(data)</code>	Tạo mảng một chiều có nhãn.
<code>s.index</code>	Trả về nhãn của Series.
<code>s.values</code>	Trả về các giá trị bên trong.
<code>df["col"]</code>	Chọn một cột dưới dạng Series.
<code>df[["c1", "c2"]]</code>	Chọn nhiều cột dưới dạng DataFrame.
<code>df.loc[label]</code>	Chọn theo nhãn dòng.
<code>df.iloc[position]</code>	Chọn theo vị trí số nguyên.

Quick Check: Series và lựa chọn

Câu 1. Pandas Series là:

- ▶ A. một bảng hai chiều
- ▶ B. một mảng một chiều có nhãn
- ▶ C. một máy chủ cơ sở dữ liệu
- ▶ D. một thư viện vẽ biểu đồ

Câu 2. Bộ chọn nào dựa trên nhãn?

- ▶ A. `iloc`
- ▶ B. `loc`
- ▶ C. `shape`
- ▶ D. `head`

Lọc dữ liệu bằng điều kiện Boolean

Lọc các dòng thỏa mãn một điều kiện.

```
high_gpa = df[df["GPA"] >= 3.5]  
print(high_gpa)
```

Diễn giải

`df["GPA"] >= 3.5` tạo một Series Boolean. Pandas giữ lại các dòng có giá trị True.

Kết quả chạy code: lọc dữ liệu

Với điều kiện GPA ≥ 3.5 , Pandas chỉ giữ lại các dòng thỏa mãn điều kiện.

```
high_gpa = df[df["GPA"] >= 3.5]  
print(high_gpa)
```

#	Name	Age	GPA
# StudentID			
# S02	Binh	21	3.6
# S04	Dung	22	3.8

Cách đọc

Hai sinh viên S02 và S04 có GPA lớn hơn hoặc bằng 3.5 nên xuất hiện trong kết quả.

Lọc với nhiều điều kiện

```
selected = df[
    (df["Age"] >= 20) &
    (df["GPA"] >= 3.5)
]
print(selected)
```

- ▶ Dùng & cho AND.
- ▶ Dùng | cho OR.
- ▶ Dùng ~ cho NOT.
- ▶ Đặt từng điều kiện trong ngoặc tròn.

Sắp xếp DataFrame

```
# sắp xếp theo ộmt ộct  
by_gpa = df.sort_values("GPA")  
  
# sắp xếp ảgim ầdn  
by_gpa_desc = df.sort_values("GPA", ascending=False)  
  
# sắp xếp theo ềnhieu ộct  
ordered = df.sort_values(  
    ["Age", "GPA"],  
    ascending=[True, False]  
)
```

Ví dụ kinh doanh

Sắp xếp khách hàng theo doanh thu giảm dần, sau đó theo tần suất đặt hàng giảm dần.

Kết quả chạy code: lọc nhiều điều kiện và sắp xếp

```
selected = df[(df["Age"] >= 20) & (df["GPA"] >= 3.5)]  
print(selected)
```

```
#           Name  Age  GPA  
# StudentID  
# S02      Binh   21  3.6  
# S04      Dung   22  3.8
```

```
print(df.sort_values("GPA", ascending=False))
```

```
#           Name  Age  GPA  
# StudentID  
# S04      Dung   22  3.8  
# S02      Binh   21  3.6  
# S03       Chi   19  3.4  
# S01       An    20  3.2
```

Các lệnh chính: lọc và sắp xếp

Lệnh	Ý nghĩa
<code>df[df["x"] > a]</code>	Giữ các dòng trong đó cột x lớn hơn a.
<code>(cond1) & (cond2)</code>	Kết hợp điều kiện bằng AND.
<code>(cond1) (cond2)</code>	Kết hợp điều kiện bằng OR.
<code>~cond</code>	Phủ định một điều kiện.
<code>df.sort_values("x")</code>	Sắp xếp dòng theo một cột.
<code>ascending=False</code>	Sắp xếp giảm dần.

Quick Check: lọc và sắp xếp

Câu 1. Biểu thức nào lọc các dòng có GPA ≥ 3.5 ?

- ▶ A. `df[df["GPA"]  $\geq$  3.5]`
- ▶ B. `df.sort_values("GPA")`
- ▶ C. `df.read_csv("GPA")`
- ▶ D. `df.merge("GPA")`

Câu 2. Vì sao cần dùng ngoặc tròn khi kết hợp nhiều điều kiện lọc?

Đọc và ghi tệp CSV

Đọc CSV

```
df = pd.read_csv("data.csv")

df = pd.read_csv(
    "data.csv",
    sep=";",
    header=0,
    encoding="utf-8"
)
```

Ghi CSV

```
df.to_csv(
    "output.csv",
    index=False
)
```

Quan trọng

`index=False` giúp tránh xuất index của DataFrame thành một cột thừa.

Đọc và ghi tệp Excel

Đọc Excel

```
df = pd.read_excel(  
    "data.xlsx",  
    sheet_name="Sheet1"  
)
```

Ghi Excel

```
df.to_excel(  
    "output.xlsx",  
    index=False  
)
```

Ghi chú giảng dạy

Excel rất phổ biến trong bối cảnh kinh doanh, nhưng sau khi nhập dữ liệu vẫn cần kiểm tra kiểu dữ liệu và giá trị thiếu.

JSON và tệp văn bản

JSON

```
df = pd.read_json("data.json")

df.to_json(
    "output.json",
    orient="records"
)
```

Tệp văn bản có phân tách

```
df = pd.read_csv(
    "data.txt",
    sep="\t"
)
```

Ý tưởng

Nhiều tệp văn bản có cấu trúc có thể được đọc bằng `read_csv()` bằng cách thay đổi ký tự phân tách.

Các lệnh chính: nhập và xuất dữ liệu

Lệnh	Vai trò
<code>pd.read_csv()</code>	Đọc tệp CSV hoặc tệp văn bản có phân tách.
<code>df.to_csv()</code>	Ghi dữ liệu ra tệp CSV.
<code>pd.read_excel()</code>	Đọc tệp Excel.
<code>df.to_excel()</code>	Ghi dữ liệu ra tệp Excel.
<code>pd.read_json()</code>	Đọc tệp JSON.
<code>df.to_json()</code>	Ghi dữ liệu ra tệp JSON.

Quick Check: nhập và xuất dữ liệu

Câu 1. Hàm nào dùng để đọc tệp CSV?

- ▶ A. `pd.read_csv()`
- ▶ B. `pd.open_csv()`
- ▶ C. `pd.load_table_only()`
- ▶ D. `df.csv_read()`

Câu 2. Tham số nào thường dùng để tránh xuất index ra tệp?

- ▶ A. `index=False`
- ▶ B. `index=True`
- ▶ C. `header=None`
- ▶ D. `drop_index=True`

Vì sao làm sạch dữ liệu quan trọng?

Dữ liệu thực tế có thể chứa:

- ▶ giá trị thiếu;
- ▶ dòng trùng lặp;
- ▶ kiểu dữ liệu không nhất quán;
- ▶ cột rỗng hoặc không liên quan;
- ▶ văn bản không thống nhất, thừa khoảng trắng hoặc khác chữ hoa/chữ thường;
- ▶ dữ liệu số bị lẫn với văn bản.

Mục tiêu

Làm sạch dữ liệu giúp tăng độ chính xác, tính nhất quán và khả năng sử dụng trước khi phân tích hoặc xây dựng mô hình.

Ví dụ dữ liệu bẩn và kết quả kiểm tra

```
df = pd.DataFrame({
    "Name": ["An ", "Binh", "Chi", "Chi"],
    "Age": [20, None, 22, 22],
    "Price": ["10", "20", "unknown", "30"]
})

print(df)
print(df.isna().sum())
```

	Name	Age	Price
0	An	20.0	10
1	Binh	NaN	20
2	Chi	22.0	unknown
3	Chi	22.0	30

```
Name      0
Age        1
Price      0
dtype: int64
```


Phát hiện giá trị thiếu

```
print(df.isna())  
print(df.isna().sum())  
  
missing_by_column = df.isna().sum()  
total_missing = df.isna().sum().sum()
```

Diễn giải

`isna()` nhận diện các ô bị thiếu; `sum()` đếm số lượng giá trị thiếu theo cột hoặc toàn bộ DataFrame.

Xóa hoặc điền giá trị thiếu

Xóa giá trị thiếu

```
rows_complete = df.dropna()  
cols_complete = df.dropna(axis=1)
```

Điền giá trị thiếu

```
df["Age"] = df["Age"].fillna(  
    df["Age"].mean()  
)  
  
df["City"] = df["City"].fillna("Unknown")
```

Câu hỏi quyết định

Nên xóa, suy diễn hay gán nhãn riêng cho giá trị thiếu? Câu trả lời phụ thuộc vào mục tiêu phân tích.

Dòng trùng lặp và kiểu dữ liệu

Dòng trùng lặp

```
duplicate_mask = df.duplicated()
duplicate_count = duplicate_mask.sum()
df = df.drop_duplicates()
```

Kiểu dữ liệu

```
print(df.dtypes)

df["Quantity"] = df["Quantity"].astype(int)

df["Price"] = pd.to_numeric(
    df["Price"],
    errors="coerce"
)
```

Làm sạch chuỗi văn bản

Các cột văn bản thường cần được chuẩn hóa.

```
df["Name"] = df["Name"].str.strip()
df["Name"] = df["Name"].str.lower()

df["City"] = df["City"].str.replace(
    "HN",
    "Hanoi"
)
```

Vấn đề thường gặp

" An ", "AN" và "an" có thể cùng chỉ một tên, nhưng là các chuỗi khác nhau nếu chưa được làm sạch.

Kết quả chạy code: làm sạch dữ liệu

```
df["Age"] = df["Age"].fillna(df["Age"].mean())  
df["Name"] = df["Name"].str.strip().str.lower()  
df["Price"] = pd.to_numeric(df["Price"], errors="coerce")  
print(df)
```

	Name	Age	Price
0	an	20.0	10.0
1	binh	21.33	20.0
2	chi	22.0	NaN
3	chi	22.0	30.0

Cách đọc

unknown được chuyển thành NaN; tên được bỏ khoảng trắng và đưa về chữ thường.

Các lệnh chính: làm sạch dữ liệu

Lệnh	Ý nghĩa
<code>df.isna()</code>	Phát hiện giá trị thiếu.
<code>df.isna().sum()</code>	Đếm giá trị thiếu.
<code>df.dropna()</code>	Xóa quan sát bị thiếu dữ liệu.
<code>df.fillna(value)</code>	Thay thế giá trị thiếu.
<code>df.duplicated()</code>	Phát hiện dòng trùng lặp.
<code>df.drop_duplicates()</code>	Xóa dòng trùng lặp.
<code>df.astype(type)</code>	Chuyển kiểu dữ liệu.
<code>pd.to_numeric()</code>	Chuyển giá trị sang dạng số.
<code>Series.str.*</code>	Áp dụng các phương thức xử lý chuỗi.

Quick Check: làm sạch dữ liệu

Câu 1. Phương thức nào xóa các dòng có giá trị thiếu?

- ▶ A. `dropna()`
- ▶ B. `fillna()`
- ▶ C. `uplicated()`
- ▶ D. `astype()`

Câu 2. Phương thức nào xóa các dòng trùng lặp?

- ▶ A. `drop_duplicates()`
- ▶ B. `dropna()`
- ▶ C. `sort_values()`
- ▶ D. `merge()`

Dữ liệu ví dụ cho thao tác và tổng hợp

```
df = pd.DataFrame({  
    "Region": ["North", "North", "South", "South", "East"],  
    "Product": ["A", "B", "A", "B", "A"],  
    "Quantity": [2, 3, 5, 4, 1],  
    "Price": [10, 20, 8, 15, 12],  
    "Sales": [20, 60, 40, 60, 12]  
})  
print(df)
```

	Region	Product	Quantity	Price	Sales
0	North	A	2	10	20
1	North	B	3	20	60
2	South	A	5	8	40
3	South	B	4	15	60
4	East	A	1	12	12

Tạo cột tính toán

```
df["Revenue"] = df["Quantity"] * df["Price"]  
  
total_revenue = df["Revenue"].sum()  
print(total_revenue)
```

Ví dụ kinh doanh

Doanh thu, biên lợi nhuận, tỷ lệ giảm giá, giá trị vòng đời khách hàng và độ trễ giao hàng thường có thể được tạo từ các cột hiện có.

Kết quả chạy code: cột tính toán

```
df["Revenue"] = df["Quantity"] * df["Price"]  
print(df[["Product", "Quantity", "Price", "Revenue"]])  
print(df["Revenue"].sum())
```

	Product	Quantity	Price	Revenue
0	A	2	10	20
1	B	3	20	60
2	A	5	8	40
3	B	4	15	60
4	A	1	12	12

192

Áp dụng hàm

Dùng map()

```
df["Status"] = df["Score"].map(  
    lambda x: "Pass" if x >= 5 else "Fail"  
)
```

Dùng apply()

```
df["Squared"] = df["Value"].apply(  
    lambda x: x ** 2  
)
```

Thực hành tốt

Ưu tiên các phép toán vector hóa trên cột khi có thể; dùng apply() khi cần logic tùy chỉnh theo dòng hoặc theo phần tử.

Chuẩn hóa và chuẩn hóa Z-score

```
# Min-max normalization
df["Normalized"] = (
    (df["Value"] - df["Value"].min()) /
    (df["Value"].max() - df["Value"].min())
)

# Z-score standardization
df["Z"] = (
    (df["Value"] - df["Value"].mean()) /
    df["Value"].std()
)
```

Diễn giải

Min-max normalization đưa dữ liệu về một khoảng cố định; Z-score biểu diễn giá trị theo đơn vị độ lệch chuẩn.

Thống kê mô tả

```
print(df.describe())

print(df["Sales"].mean())
print(df["Sales"].median())
print(df["Sales"].min())
print(df["Sales"].max())
print(df["Sales"].std())
```

Vai trò phân tích

Thống kê mô tả giúp nhận biết thang đo, xu hướng trung tâm, độ biến thiên và các giá trị bất thường có thể tồn tại.

Gom nhóm với groupby ()

```
region_total = df.groupby("Region")["Sales"].sum()
print(region_total)

region_stats = (
    df.groupby("Region")["Sales"]
      .agg(["count", "sum", "mean", "min", "max"])
)
print(region_stats)
```

Split-apply-combine

Pandas chia dữ liệu thành các nhóm, áp dụng phép tổng hợp cho từng nhóm, sau đó kết hợp kết quả.

Kết quả chạy code: groupby và agg

```
region_total = df.groupby("Region")["Sales"].sum()  
print(region_total)
```

```
Region  
East      12  
North     80  
South    100  
Name: Sales, dtype: int64
```

```
region_stats = df.groupby("Region")["Sales"].agg(["count", "sum", "mean"])  
print(region_stats)
```

	count	sum	mean
Region			
East	1	12	12.0
North	2	80	40.0
South	2	100	50.0

Gom nhóm theo nhiều cột

```
summary = (  
    df.groupby(["Region", "Product"])["Sales"]  
        .sum()  
)  
print(summary)
```

Diễn giải kinh doanh

Cách này giúp trả lời các câu hỏi như: sản phẩm nào đóng góp doanh thu lớn nhất tại từng khu vực?

Các lệnh chính: thao tác và tổng hợp

Lệnh	Ý nghĩa
<code>df["new"] = ...</code>	Tạo hoặc biến đổi một cột.
<code>Series.map()</code>	Ánh xạ giá trị sang giá trị mới.
<code>Series.apply()</code>	Áp dụng hàm lên các phần tử của Series.
<code>df.describe()</code>	Thống kê mô tả.
<code>df.groupby()</code>	Gom nhóm quan sát.
<code>.agg()</code>	Áp dụng nhiều hàm tổng hợp.
<code>mean(), median(), std()</code>	Các thống kê số thường dùng.

Quick Check: thao tác dữ liệu

Câu 1. Phương thức nào dùng để gom nhóm quan sát theo category?

- ▶ A. `groupby()`
- ▶ B. `dropna()`
- ▶ C. `astype()`
- ▶ D. `sort_index()`

Câu 2. Hoàn thành công thức: `Revenue = Quantity ... Price`.

Nối DataFrame bằng concat

```
combined = pd.concat(  
    [df1, df2],  
    axis=0  
)
```

- ▶ `axis=0`: xếp chồng các dòng theo chiều dọc.
- ▶ `axis=1`: ghép các cột theo chiều ngang.

Ví dụ

Nối dữ liệu bán hàng tháng 1 và tháng 2 khi hai bảng có cùng cấu trúc cột.

Dữ liệu ví dụ cho merge

```
customers = pd.DataFrame({  
    "CustomerID": [1, 2, 3],  
    "Name": ["An", "Binh", "Chi"]  
})
```

```
orders = pd.DataFrame({  
    "OrderID": [101, 102, 103],  
    "CustomerID": [1, 1, 2],  
    "Amount": [200, 150, 300]  
})
```

Mục tiêu

Ghép thông tin khách hàng với thông tin đơn hàng thông qua khóa chung CustomerID.

Ghép bảng bằng merge

```
result = pd.merge(  
    customers,  
    orders,  
    on="CustomerID",  
    how="inner"  
)
```

- ▶ inner: chỉ giữ các bản ghi khớp nhau.
- ▶ left: giữ toàn bộ bản ghi từ bảng bên trái.
- ▶ right: giữ toàn bộ bản ghi từ bảng bên phải.
- ▶ outer: giữ toàn bộ bản ghi từ cả hai bảng.

Kết quả chạy code: merge

```
result = pd.merge(customers, orders, on="CustomerID", how="inner")  
print(result)
```

	CustomerID	Name	OrderID	Amount
0	1	An	101	200
1	1	An	102	150
2	2	Binh	103	300

Cách đọc

Khách hàng Chi không xuất hiện trong inner merge vì không có đơn hàng khớp trong bảng orders.

Pivot table

Pivot table tóm tắt các giá trị số theo các nhóm phân loại.

```
pivot = pd.pivot_table(  
    sales,  
    values="Revenue",  
    index="Region",  
    columns="Product",  
    aggfunc="sum"  
)  
print(pivot)
```

Diễn giải

Các dòng là khu vực, các cột là sản phẩm, và mỗi ô chứa tổng doanh thu.

Kết quả chạy code: pivot table

Ví dụ nếu dữ liệu có doanh thu theo khu vực và sản phẩm, pivot table có thể có dạng:

Product	A	B
Region		
East	12.0	NaN
North	20.0	60.0
South	40.0	60.0

Cách đọc

Ô tại dòng South, cột B bằng 60, nghĩa là tổng doanh thu của sản phẩm B ở khu vực South là 60.

Định dạng lại dữ liệu: pivot và melt

Từ long sang wide

```
wide = df.pivot(  
    index="Date",  
    columns="Product",  
    values="Sales"  
)
```

Từ wide sang long

```
long = pd.melt(  
    wide.reset_index(),  
    id_vars="Date"  
)
```

Ý tưởng

Dạng dữ liệu nên phù hợp với nhiệm vụ phân tích: trực quan hóa, tổng hợp, mô hình hóa hoặc báo cáo.

Các lệnh chính: kết hợp và định dạng lại

Lệnh	Ý nghĩa
<code>pd.concat()</code>	Kết hợp DataFrame theo một trục.
<code>pd.merge()</code>	Kết hợp DataFrame dựa trên cột khóa.
<code>how="inner"</code>	Chỉ giữ các bản ghi khớp.
<code>how="left"</code>	Giữ toàn bộ bản ghi từ bảng trái.
<code>df.pivot()</code>	Chuyển dữ liệu dạng long sang wide.
<code>pd.melt()</code>	Chuyển dữ liệu dạng wide sang long.
<code>pd.pivot_table()</code>	Tạo bảng tóm tắt.

Quick Check: kết hợp và định dạng lại

Câu 1. Hàm nào kết hợp các DataFrame bằng một cột khóa?

- ▶ A. `pd.merge()`
- ▶ B. `pd.mean()`
- ▶ C. `pd.reshape()`
- ▶ D. `pd.filter_rows()`

Câu 2. Pivot table dùng để làm gì?

Tương quan

Tương quan đo mức độ liên hệ giữa các biến số.

```
print(df.corr(numeric_only=True))  
  
print(  
    df["Advertising"].corr(df["Sales"])  
)
```

Quan trọng

Tương quan không tự động chứng minh quan hệ nhân quả.

Kết quả chạy code: tương quan và chuỗi thời gian

```
print(df[["Sales", "Quantity", "Price"]].corr())
```

	Sales	Quantity	Price
Sales	1.000000	0.620174	0.594089
Quantity	0.620174	1.000000	-0.272772
Price	0.594089	-0.272772	1.000000

Cách đọc

Ma trận tương quan đối xứng. Giá trị gần 1 biểu thị quan hệ tuyến tính dương mạnh hơn; tuy nhiên tương quan không chứng minh quan hệ nhân quả.

Trực quan hóa nhanh bằng Pandas

Các hàm vẽ của Pandas được xây dựng trên Matplotlib.

```
df.plot(x="Month", y="Sales", kind="line")
```

```
df.plot(x="Product", y="Sales", kind="bar")
```

```
df["Sales"].plot(kind="hist")
```

```
df.plot(x="Advertising", y="Sales", kind="scatter")
```

Quy trình xử lý chuỗi thời gian

```
df["Date"] = pd.to_datetime(df["Date"])  
df = df.set_index("Date")  
df = df.sort_index()  
  
weekly = df["Sales"].resample("W").sum()  
monthly = df["Sales"].resample("ME").sum()
```

Ý tưởng chuỗi thời gian

Sau khi chuyển cột ngày và đặt làm index, Pandas có thể tổng hợp dữ liệu theo các chu kỳ lịch.

Thành phần ngày tháng và trung bình trượt

Thành phần ngày tháng

```
df["Year"] = df.index.year  
df["Month"] = df.index.month  
df["Day"] = df.index.day
```

Trung bình trượt

```
df["MovingAvg"] = (  
    df["Sales"]  
        .rolling(window=3)  
        .mean()  
)
```


Các lệnh chính: thao tác nâng cao

Lệnh	Ý nghĩa
<code>df.corr()</code>	Tính ma trận tương quan.
<code>Series.corr()</code>	Tính tương quan giữa hai Series.
<code>df.plot()</code>	Trực quan hóa nhanh.
<code>pd.to_datetime()</code>	Chuyển dữ liệu sang kiểu datetime.
<code>df.set_index()</code>	Đặt một cột làm index.
<code>df.resample()</code>	Tổng hợp dữ liệu chuỗi thời gian theo khoảng thời gian.
<code>Series.rolling()</code>	Tính thống kê cửa sổ trượt.

Quick Check: thao tác nâng cao

Câu 1. Hàm nào chuyển một cột sang kiểu datetime?

- ▶ A. `pd.to_datetime()`
- ▶ B. `pd.date_convert_only()`
- ▶ C. `df.datetime()`
- ▶ D. `pd.time_series()`

Câu 2. Phương thức nào tính ma trận tương quan?

- ▶ A. `corr()`
- ▶ B. `merge()`
- ▶ C. `dropna()`
- ▶ D. `pivot()`

Tóm tắt nội dung I

Chủ đề	Ý chính
Pandas	Thư viện Python để xử lý và phân tích dữ liệu có cấu trúc.
Series	Mảng một chiều có nhãn.
DataFrame	Bảng dữ liệu hai chiều có nhãn.
Kiểm tra dữ liệu	Dùng <code>head()</code> , <code>shape</code> , <code>info()</code> và <code>describe()</code> .
Lựa chọn dữ liệu	Dùng ngoặc chọn cột, <code>loc</code> và <code>iloc</code> .
Lọc dữ liệu	Dùng điều kiện Boolean để giữ các dòng phù hợp.

Tóm tắt nội dung II

Chủ đề	Ý chính
Nhập/xuất dữ liệu	Đọc và ghi CSV, Excel, JSON và tệp văn bản.
Làm sạch	Xử lý giá trị thiếu, trùng lặp, kiểu dữ liệu và chuỗi.
Thao tác	Tạo cột tính toán và biến đổi dữ liệu.
Tổng hợp	Dùng groupby() và agg() để tạo báo cáo tóm tắt.
Kết hợp	Dùng concat() và merge() cho nhiều bảng dữ liệu.
Nâng cao	Tương quan, trực quan hóa và thao tác chuỗi thời gian.

Thực hành: dữ liệu sinh viên

Bài tập 1

Tạo một DataFrame có ít nhất 10 sinh viên với các cột:

- ▶ StudentID, Name, Age, Major, GPA.

Thực hiện:

- 1 kiểm tra bằng `head()`, `shape`, `info()` và `describe()`;
- 2 lọc các sinh viên có `GPA >= 3.5`;
- 3 sắp xếp theo GPA giảm dần;
- 4 chọn dòng bằng `loc` và `iloc`.

Thực hành: dữ liệu thiếu và làm sạch

Bài tập 2

Thêm giá trị thiếu vào tập dữ liệu sinh viên. Sau đó:

- 1 đếm số giá trị thiếu;
- 2 điền Age thiếu bằng Age trung bình;
- 3 điền GPA thiếu bằng GPA trung vị;
- 4 xóa các dòng thiếu Name.

Bài tập 3

Tạo một dữ liệu lộn xộn có khoảng trắng trong tên, dòng trùng lặp và giá trị số bị lẫn văn bản. Làm sạch bằng `str.strip()`, `drop_duplicates()` và `pd.to_numeric()`.

Thực hành: phân tích bán hàng

Bài tập 4

Tạo một tập dữ liệu bán hàng gồm Date, Region, Product, Quantity và Price. Tạo:

$$\text{Revenue} = \text{Quantity} \times \text{Price}.$$

Sau đó tính:

- 1 tổng doanh thu;
- 2 doanh thu theo Region;
- 3 doanh thu theo Product;
- 4 doanh thu trung bình theo Region;
- 5 pivot table Region $\times$ Product.

Thực hành: merge và chuỗi thời gian

Bài tập 5: Merge

Tạo `customers` (`CustomerID`, `Name`, `City`) và `orders` (`OrderID`, `CustomerID`, `Amount`). Thực hiện inner merge, left merge và xác định khách hàng chưa có đơn hàng.

Bài tập 6: Chuỗi thời gian

Tạo dữ liệu bán hàng ngày trong ít nhất 30 ngày. Chuyển `Date` sang `datetime`, đặt làm index, tính tổng doanh thu theo tuần, tính trung bình trượt 7 ngày và vẽ line plot.

Câu hỏi phản tư cuối bài

- ▶ Series và DataFrame khác nhau như thế nào?
- ▶ Khi nào nên dùng loc và khi nào nên dùng iloc?
- ▶ Vì sao cần kiểm tra shape, dtypes và giá trị thiếu từ sớm?
- ▶ concat() khác merge() như thế nào?
- ▶ Vì sao groupby() quan trọng trong phân tích kinh doanh?
- ▶ Vì sao tương quan không tự động chứng minh quan hệ nhân quả?

Đáp án gợi ý cho Quick Check

Phần	Đáp án
Pandas là gì?	B; C
Cơ bản về DataFrame	A; A
Series và lựa chọn	B; B
Nhập và xuất dữ liệu	A; A
Làm sạch dữ liệu	A; A
Thao tác dữ liệu	A; Revenue = Quantity $\times$ Price
Kết hợp và định dạng lại	A; tóm tắt giá trị theo category
Thao tác nâng cao	A; A